

day 21

🛡️ Day 21: NestJS Authentication, JWT & Architecture

1. Authentication Concepts in NestJS

في (Authentication) الاحترافية، نقوم بفصل عملية المصادقة REST API في تطبيقات `AuthModule` مديول مستقل يسمى.

🔄 Authentication (JWT Flow) دورة حياة الـ

1. ****Register/Login:**** يرسل العميل البريد الإلكتروني وكلمة المرور.
2. ****Validation & Hashing:**** يتم التحقق من المستخدم وتشفير كلمة المرور بـ `bcrypt`.
3. ****Token Generation:**** باستخدام `Access Token` يقوم السيرفر بإنشاء `@nestjs/jwt`.
4. ****Authorized Requests:**** كـ Header في الـ Token يرسل العميل الـ `Protected Routes` للوصول للـ `Authorization: Bearer <token>`.

2. Key Modules & Services

🔑 JwtModule (@nestjs/jwt)

JSON Web Tokens لإنشاء والتحقق من الـ `AuthModule` يتم تسجيله في

```typescript

```
JwtModule.registerAsync({
 imports: [ConfigModule],
 useFactory: async (configService: ConfigService) => ({
 secret: configService.get<string>('JWT_SECRET'),
 signOptions: { expiresIn: '1d' },
 }),
 inject: [ConfigService],
})
```

## 3. Protecting Routes using Guards (@UseGuards)

الـ **Guard** هو كلاس يطبق واجهة `CanActivate` لتحديد ما إذا كان المسموح للطلب بالمرور أم لا.

### 🛡️ AuthGuard Example:

TypeScript

```

@Inject()
export class AuthGuard implements CanActivate {
 constructor(private jwtService: JwtService) {}

 async canActivate(context: ExecutionContext): Promise<boolean> {
 const request = context.switchToHttp().getRequest();
 const token = this.extractTokenFromHeader(request);
 if (!token) throw new UnauthorizedException();

 try {
 const payload = await this.jwtService.verifyAsync(token);
 request['user'] = payload; // إرفاق بيانات المستخدم بالطلب
 } catch {
 throw new UnauthorizedException();
 }
 return true;
 }
}

```

## Project Directory Architecture (Day 2)

Plaintext

```

src/
├── auth/ # مديول التوثيق وتوليد الـ Tokens
│ ├── dto/ # Auth DTOs (LoginDto, RegisterDto)
│ ├── auth.controller.ts
│ ├── auth.module.ts
│ └── auth.service.ts
├── common/ # العناصر المشتركة (Guards, Decorators, Filters)
│ ├── decorators/
│ ├── guards/
│ └── auth.guard.ts
├── user/ # إدارة بيانات المستخدمين
└── post/ # إدارة المقالات

```